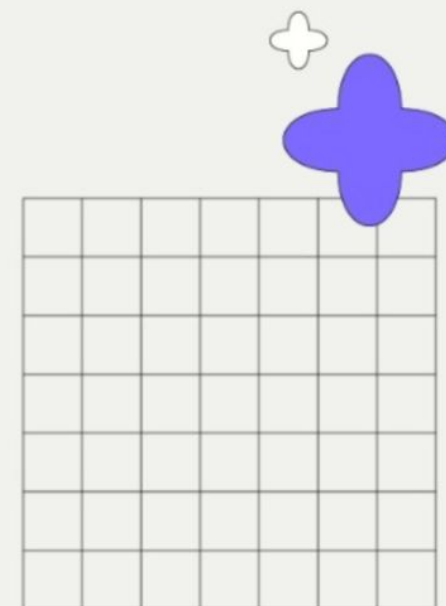
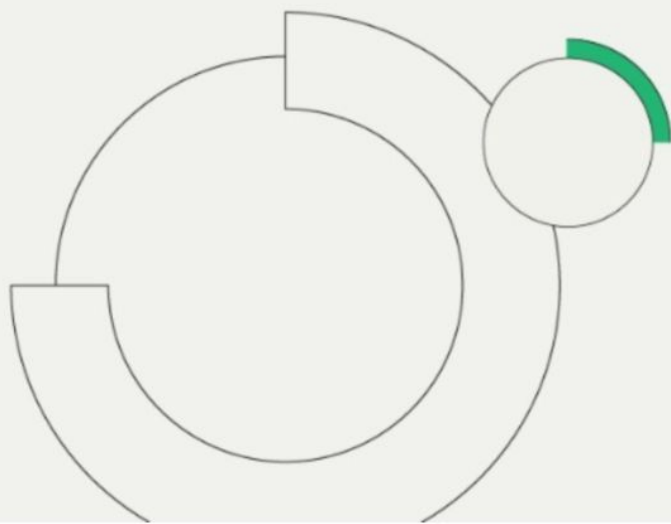
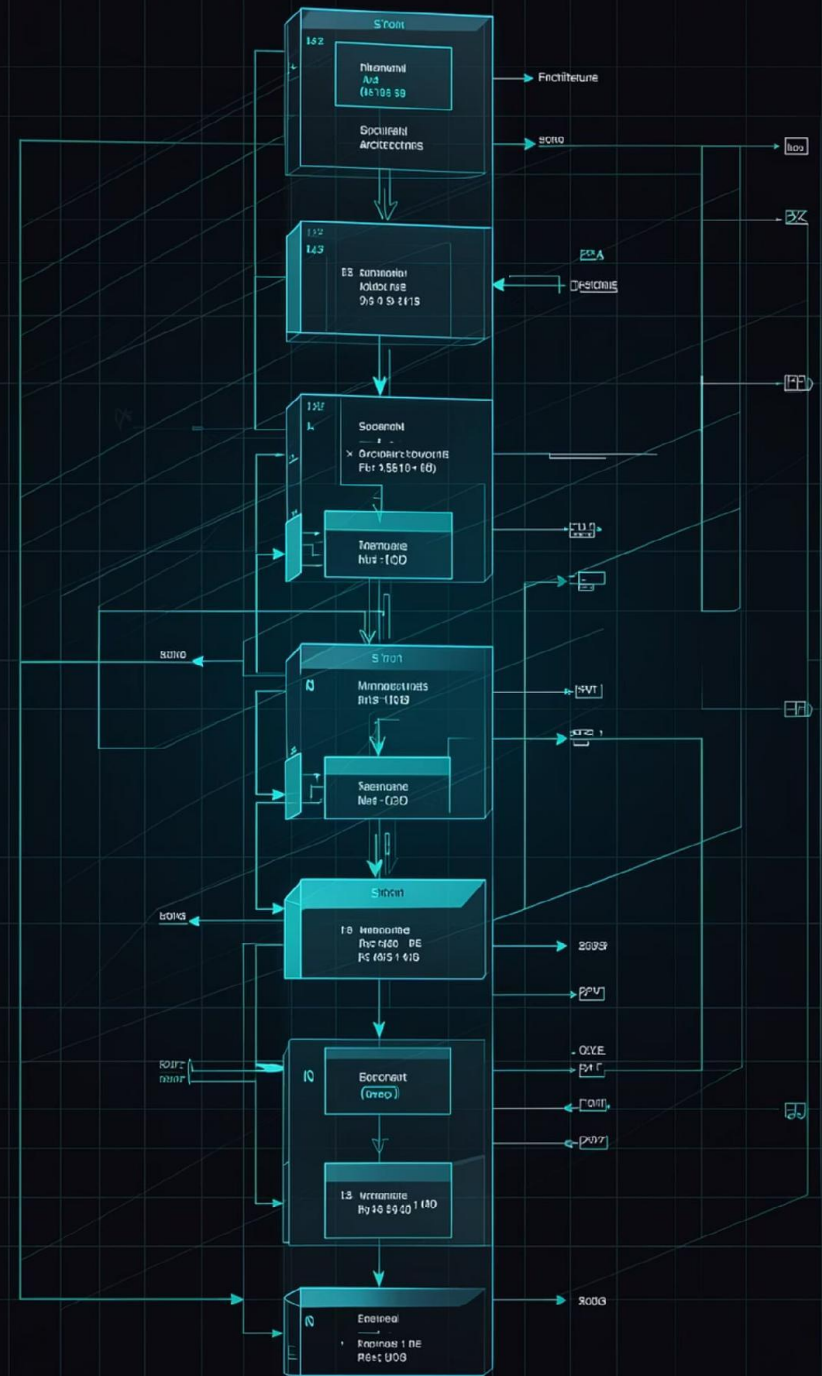




שיעור 12 - Vibe Engineering

שיעור 12: Vibe Coding- להנדסת מערכת Vibe Engineering

קורס מיישם AI, האקריו





נקודת המעבר של הקורס

מכאן

הנדסת מערכת
וארכיטקטורה

עד עתה

Vibe Coding
MLP
הוכחנו ערך מהר

סיפור ה-75 דולר

רועי בנה מערכת מעולה באמצעות `claude code`.
אחרי חודש הגיע חשבון- $75\$$, ולא ברור על מה.

זה ההבדל בין בנייה נכונה של מערכת מידע ותכנון
נכון של הארכיטקטורה
לבנייה של MLP- ואב טיפוס
חוסר תכנון יוצר בסוף חשבון בסוף החודש.

הסיפור האמיתי מאחורי הקלעים

רועי אינו מתכנת. את המערכת בנה עבורו סוכן קוד (Claude Code) ובחר כברירת מחדל כלים מושלמים



FastAPI



Railway



SQLite



Claude Sonnet

כולם כלים מצוינים ומקצועיים - אבל אף אחד מהם לא חינמי.
רועי אישר, בלי להבין מה קיים בקוד.



שירות שדולק 24/7 מול שירות שנרדם

חדר מלון 🏠

משלמים על כל זמן השכירות על החדר
גם כשהחדר ריק לגמרי.

מונית 🚕

משלמים רק על הנסיעה, כשלא עולים
על המונית, לא משלמים

בשלב הזה אנחנו נבחר **רק שירותים שנרדמים**: Serverless, Apps Script, אירוח סטטי.

מה יכול לייצר חיובים מפתיעים?



קריאות API בתשלום
כל קריאה לספק חיצוני שעולה
כסף



מסלול התנסות שמסתיים והופך
למסלול בתשלום.
בלי התראה מוקדמת, הכרטיס
כבר במערכת



שירות ענן שנשאר דולק!
כמו חדר בבית מלון

נמנעים משלושתם באותה דרך: שואלים את ה-AI מה חינמי, ואם אין ברירה, מגדירים תקרה.

כלל 1: עלינו לברר מי גובה מה ומתי

אסור שירוך שום רכיב במערכת שאיננו יודעים מי גובה עליו כסף ומתי.

לפני חיבור של כל שירות - נשאל שלוש שאלות:

3

האם יש תקרת הוצאה
שניתן להגדיר?

2

מה קורה כשחוצים אותה?

1

האם יש שכבה חנימית ומה
היא כוללת?

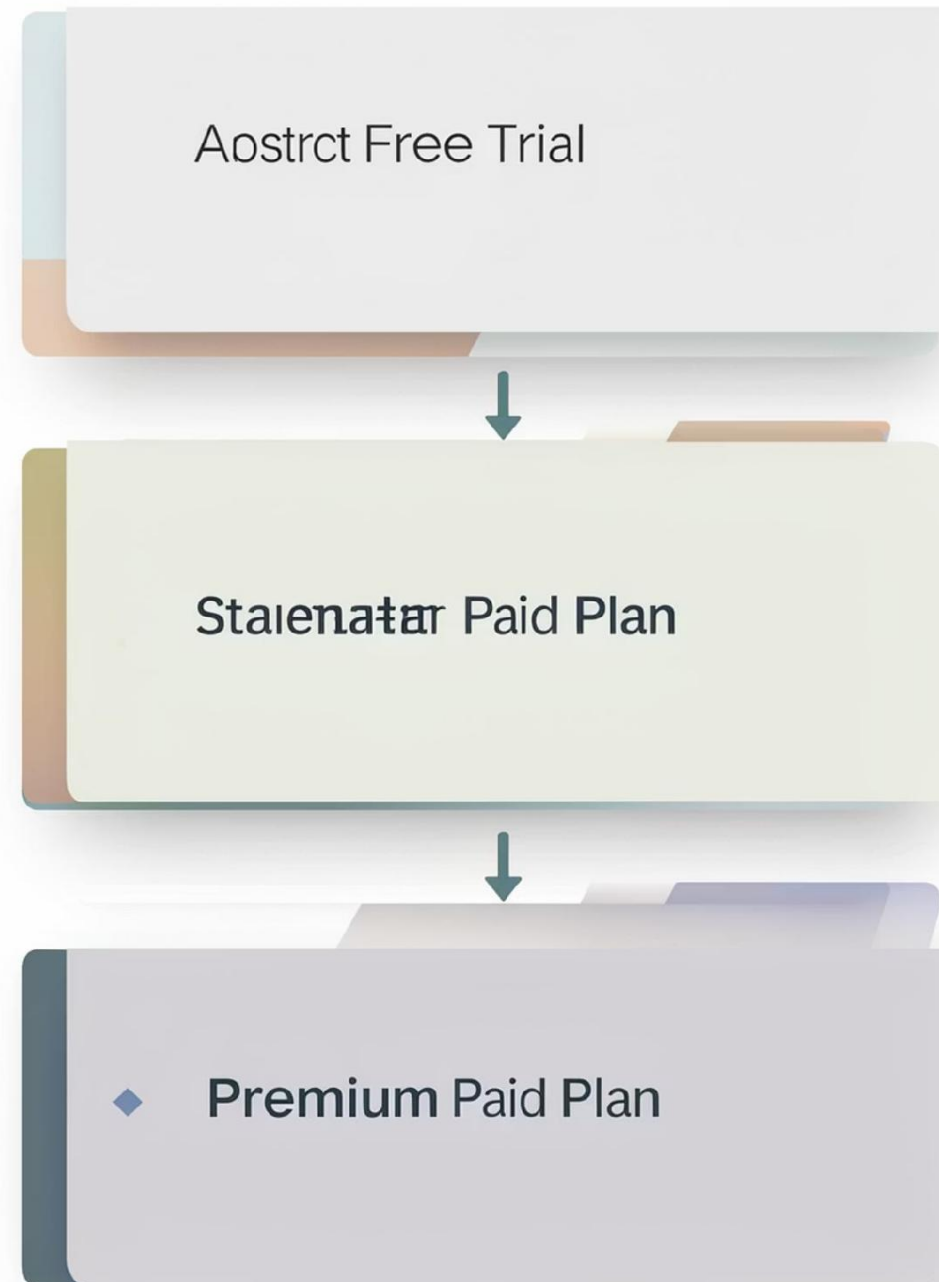
שלושה סוגים חינמיים

	חינמי תמיד - בגבולות שימוש בטוח, בלי כרטיס אשראי, נעצר במקום לחייב
--	---

	ניסיון חינם או קרדיט כאן נולדים החשבונות המפתיעים כשהחינם מסתיים והכרטיס כבר קיים במערכת
--	--

	משולם מהשקל הראשון צפוי, בתנאי שהגדרנו תקרה
--	--

בקורס נשתמש רק בסוג הראשון, נלמד על היתר.



הכלי החינמי - Apps Script

FE, BE I-DB

הכול ב-Google Apps Script. קובץ HTML
(מציג), קובץ קוד (מחליט), גיליון (זוכר).

פרויקט אחד

חשבון אחד, כתובת אחת.

AI

בשכבה החינמית Gemini API.

גרסאות

מובנות ב-Apps Script. כל פריסה היא גרסה
שמורה.

מה נוכל לעשות בלי ידע טכני?

נשאל את ה AI שלוש שאלות:



החזית - FE

איפה נמצא מה שמוצג
למשתמש?
מה המנגנון שמציג?



המוח - BE

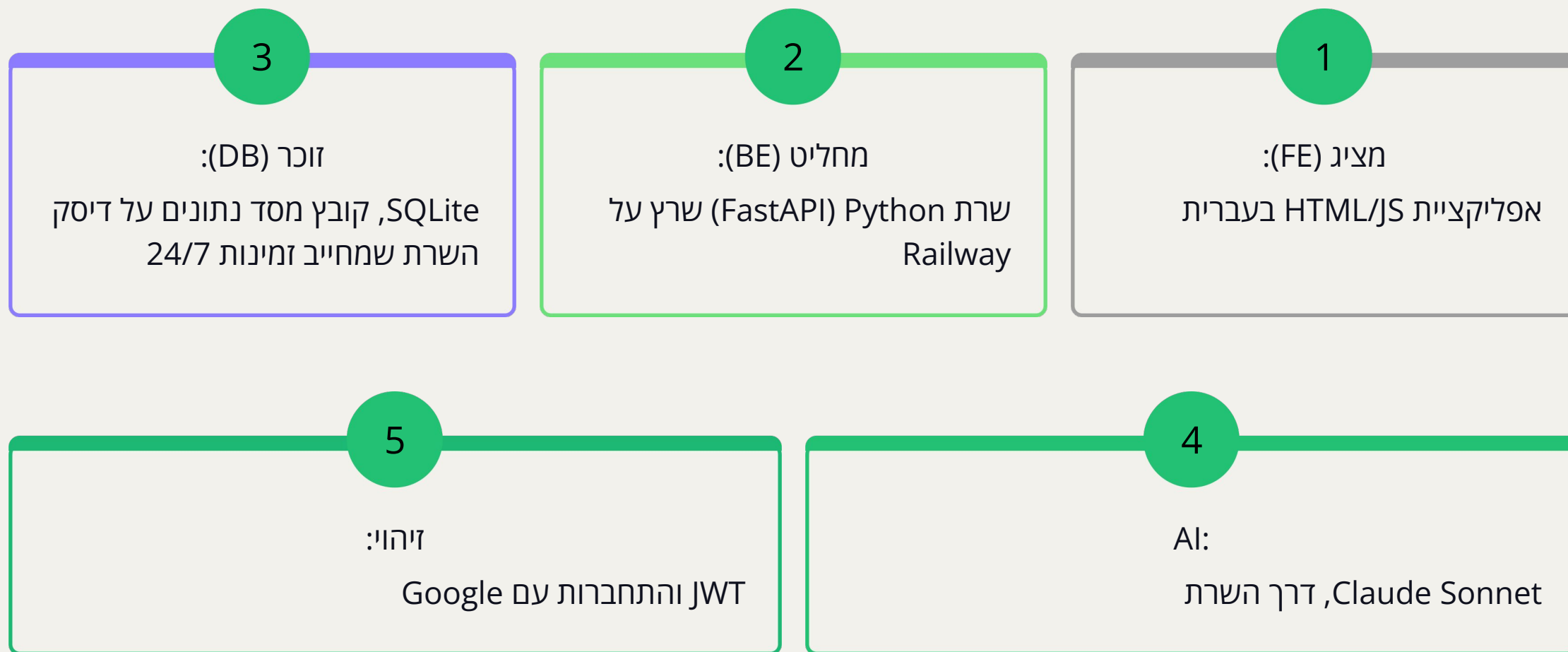
איפה שומרים את המנגנון
שחושב ומחליט?



הזיכרון - DB

איפה שומרים את כל המידע?
מה המנגנון שזוכר?

הארכיטקטורה שהסוכן בנה, בשפה שלנו



מה הסוכן עשה נכון

הסוכן בנה נכון!
כמו שאמרנו -
הבנייה היא לא החסם

הגבלת שימוש יומית וחודשית
לכל משתמש.
(כלל 1, ברמת המשתמש הבודד)

כלל 1

1

2

3

4

הפרדה מלאה לשלוש שכבות

המפתח בצד השרת בלבד,
כמשתנה סביבה.
אף פעם לא בדפדפן. (כלל 2)

כלל 2

אז מאיפה הגיע החשבון?

חשוד 2: Railway

מריץ את השרת 24 שעות ביממה,
גם כשאיש לא מחובר.

חשוד 1: Claude Sonnet

משלמים מהטוקן הראשון. אין לו
שכבה חינמית.

שורש הבעיה - למה השרת חייב לדלוק?
כי SQLite הוא קובץ שחי על הדיסק של השרת.
בחירת ה-DB כבלה את המערכת להיות זמינה
24/7 כמו החדר בבית המלון.

התובנה המרכזית

הסוכן של AI מציע ובוחר
כברירת מחדל כלים
מקצועיים טובים ומוכרים,
אבל לא את החינמיים

הוא מחיר של הכרעות
שאנחנו לא יודעים שקיבלנו

החשבון הוא לא מחיר של
בורות

כשלא שואלים אותו "כמה זה יעלה בחודש", הוא לא יתייחס לתקציב
זו שאלה שסוכן עצמו לא מעלה.

מה זה מלמד אותנו?

AI יודע איך לבנות. עלינו ללמוד לפקח עליו.

- מציג FE ← מחליט BE ← זוכר DB
- דולק מול נרדם
- שלושת סוגי ה"חינם"

שלוש שאלות לכל סוכן לפני שמאשרים:

- איפה זה ירוץ, מי משלם?
- יש שכבה חינומית?
- מה החלופה הזולה שכדאית לצרכים שלנו?

הסוכן עונה מצוין על שלושתן. הוא רק צריך להישאל.



המודל מציע, האדם מכריע

אצל רועי:

- המודל הציע
- המודל הכריע.
- הוא לא הציג שנדרשת הכרעה.

איך אנחנו יודעים שזה נדרש? לא יודעים.

השיעור הזה ממקד אותנו בשאלות הללו

מעכשיו: אתם מכריעים.



דוגמה ממערכת דיווח תקלות



זוכר (DB)

- שמירת הדיווחים
- הזהויות המוצפנות
- הנתונים הגולמיים והמעובדים



מחליט (BE)

- ניתוח הדיווח על ידי AI
- סיווג דחיפות
- בדיקת הרשאות



מציג (FE)

- מסך דיווח התקלה
- הדשבורד
- המונה האישי

כל יכולת מתפצלת לכל השכבות הרלוונטיות.
לדוגמה, "דיווח תקלה" מתפצל לשלוש השכבות:
הטופס (מציג), הניסוח והסיווג (מחליט), והרשומה (זוכר).





כלל 2: כל דבר שייך לשכבה אחת

או מציג (FE), או מחליט (BE), או זוכר (DB).

מפתח API ב-BE

מפתח API הוא סוד.
סוד שייך ל"מחליט".
מקומו בשרת בלבד.

מפתח בדפדפן

גלוי לכל מי שפותח כלי
מפתחים. הדרך
הקצרה ל-75 דולר.

לכל מסך ב MLP שלכם נתחו מה עובר ל BE ומה ל DB



תרגיל 1: בקבוצות - טבלת החלוקה

כל קבוצה לוקחת את מסכי אב הטיפוס שלה ומנתחת:

יכולת	שכבה	כיצד לפצל אותם לאילו חלקים?
הצגת מסך דיווח	FE	-
ניסוח AI לדיווח	BE	קריאה ל-API, עיבוד תשובה
שמירת דיווח	DB	-
בדיקת הרשאות	BE	-

צרו טבלה לכל היכולות

כל יכולת - לאיזו שכבה שייכת, ובפיצול מתפצלת - לאילו חלקים.

Team Analysis

Team Analysis	Entdmioce			4	3	Mean
Droonalvis						Pronisrnoice
Pleam Amciss	Side	Penus	Sluine	Drif	sis	Distuuti
Eccdiniae						
Jcfounind						
Mecdunool'e						
Homsle						
Comode						

הכלים החינמיים לכל שכבה



החזית - FE

GitHub Pages / Netlify / Vercel
אירוח סטטי חינמי לחלוטין



המוח - BE

Google Apps Script או Serverless בשכבה חינמית
נדלק רק כשקוראים לו



הזיכרון - DB

Google Sheets בשלב הזה
פשוט, שקוף, חינמי



הבינה - AI

Gemini API בשכבה החינמית
בלי כרטיס אשראי



Gemini בשכבה החינמית

- חלק מחשבון גוגל שלנו
- חינמי, בלי כרטיס אשראי
- מגבלת קצב: הגבלה לכמות הבקשות לדקה וליום
- חוצים את המגבלה? הבקשות נחסמות עד מחר, **בלי חיוב**

המסקנה

כל המערכת, מקצה לקצה, רצה באפס שקלים.

גם מפתח חינמי הוא סוד

מפתח ה-Gemini שייך ל-BE לפי כלל 2.

ההרגל נבנה עכשיו, גם כשטעות לא עולה כסף, כדי שיהיה מובן מאליו כשתעלה.



הרגלי אבטחה נכונים נבנים בסביבה בטוחה - ומתגלגלים אוטומטית לסביבת הייצור.

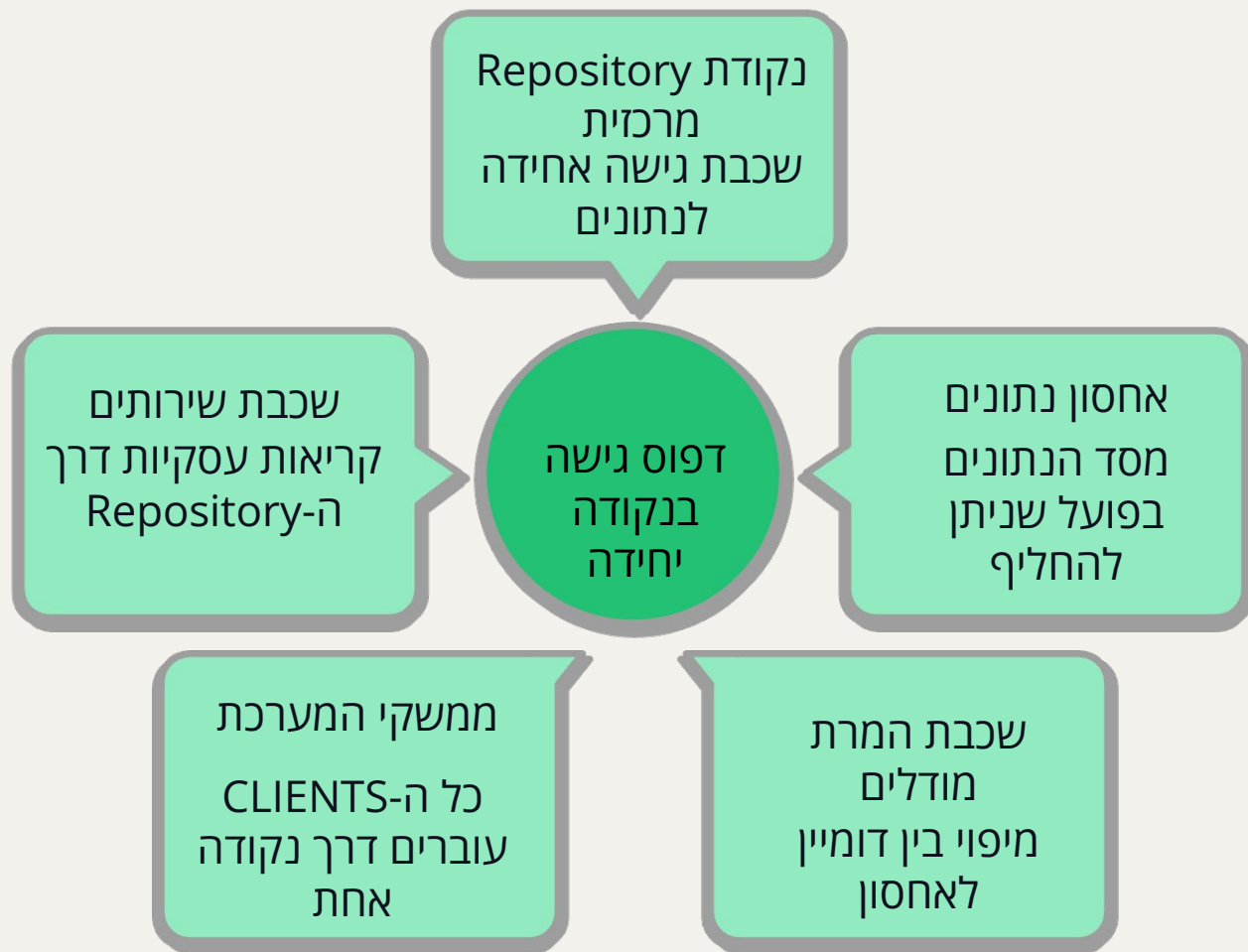
כלל 3: גישה לנתונים דרך מקום אחד - repository

הגישה לנתונים עוברת דרך מקום אחד.

זה מה שהופך את בחירת האחסון
מהחלטה גורלית להחלטה הפיכה.

כשכל הגישה לנתונים עוברת דרך נקודה
אחת:

ניתן להחליף את מסד הנתונים בשינוי של
קובץ אחד, בלי לגעת בשאר המערכת.



נשתמש ב Google Sheets קודם

מתחבר בקלות אינטגרציה פשוטה עם Apps Script ועם כל הכלים	☐	שקוף לחלוטין כל דיווח שורה, כל שדה עמודה - רואים את הנתונים בעיניים	☐	חינמי לגמרי אפס עלות, אפס הפתעות בחשבון. בנוי כמו בסיס נתונים	☐
--	--------------------------	--	--------------------------	--	--------------------------

בשיעורים הבאים: נבדוק את ה-Repository שלנו
נחליף את הגיליון במסד נתונים אמיתי בשינוי של קובץ אחד.

עכשיו תורכם

נבדוק מה יעלה לנו כסף בהמשך - בדיקת \$75



עכשיו תורכם - בדיקת ה-75 דולר

כל קבוצה עונה על השאלה:

האם אקבל חשבון בעוד חודש? ממי הוא יגיע ועל מה?

אם התשובה ברורה ללא היסוס 

הבנתם את השיעור. ממשיכים קדימה.

אם קיים היסוס 

חוזרים לצעד 2 - טבלת הכלים והעלויות.

סיכום: שלושת הכללים

כלל 1



אסור שירוצן רכיב שאינכם יודעים מי גובה עליו ומתי

כלל 2



כל דבר שייך לשכבה ספציפית: מציג, מחליט או זוכר. סודות תמיד ב-BE

כלל 3

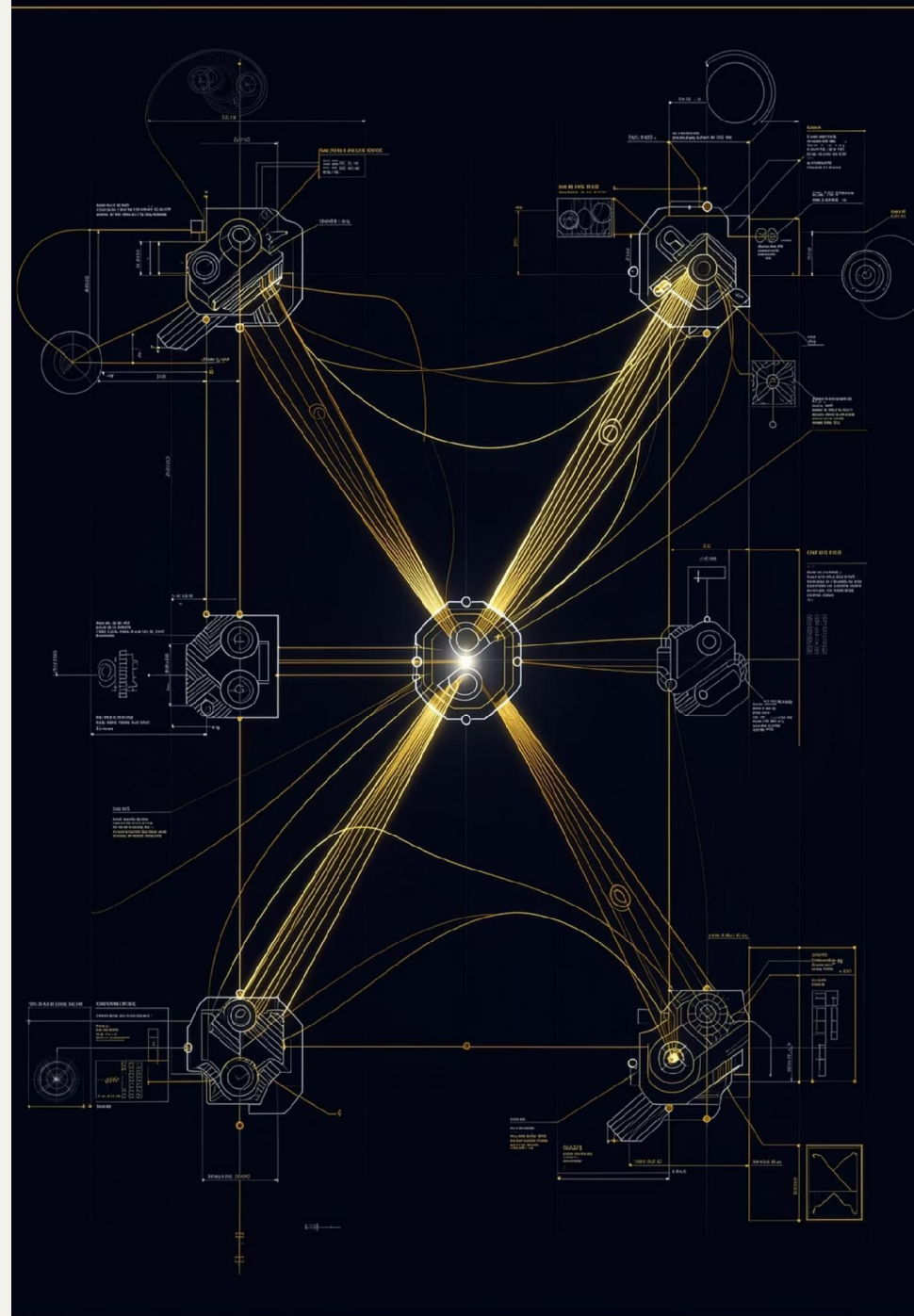


הגישה לנתונים דרך מקום אחד בלבד repository

ממקרי שימוש להחלטות פיתוח

מערכת דיווח תקלות ללא פחד

מהתרחישים שכבר כתבנו אל ההכרעות שקובעות מה
בונים, ממה, ובאיזה מחיר





איפה אנחנו במסע

1

שיעור 12: סגירת שלב
ההוכחה
אב טיפוס מאומת, שפת
שלוש השכבות, וסט הכלים
החינמי.

2

אב טיפוס מאושר
פותח את שלב החלטות
הפיתוח.

3

חלק ג של ה-PRD
ארכיטקטורת מערכת, והשער אליו הוא סעיף 14: מקרי
שימוש ותהליכים.

על מה מתבססים: לא כותבים תרחישים מאפס

הגישה הישנה

סיפור משתמש כלל שלושה חלקים:
as a - פרסונה

I WANT - הפעילות הנדרשת
SO THAT - התוצאה/ הסיבה

לזה מוסיפים קריטריוני הצלחה.

מקרה שימוש היה מסמך שנכתב מאפס: שחקן, תנאים מוקדמים, זרימה ראשית, חלופות וחריגות, ואיכויות.

המתודולוגיה שלנו

התרחישים כבר קיימים: הם נכתבו במרכיב תרחישים של סיפורי המשתמש המורחבים.

סעיף 14 ב-PRD מוסיף רק את מה שאינו קיים בסיפורים: החלטות הפיתוח.

מקרה השימוש הוא הרחבה הנדסית לסיפור המשתמש, שכבה מעל התרחישים, ולא כתיבה חוזרת שלהם.

מימוש: שלוש השכבות

FE: החזית

איפה זה מוצג?

BE: המוח

איפה זה מחליט?

DB: הזיכרון

איפה זה נשמר?

יכולת שעונה על יותר משאלה אחת מתפצלת.



דוגמה: "דיווח תקלה" = טופס (מציג) + ניסוח וסיווג (מחליט) + רשומה (זוכר). שפת השכבות היא אוצר המילים שבו נכתבות החלטות הפיתוח של סעיף 14.

תזכורת: התרחישים שכבר יש לנו

סיפור משתמש 1 (אנונימיות המדווח) כולל ארבעה תרחישים כתובים:

2

תרחיש הקצה
דיווח על טעות עצמית, והנוהל
מתוקן בלי ששום אדם נקרא
לשיחה.

1

תרחיש ההצלחה הבסיסי
המפעיל מדווח על קול חריג
במכונה, אנונימית, בפחות
מדקה.

4

תרחיש גבול המערכת
אירוע בטיחות חמור שמחייב
זיהוי לפי רגולציה, באישור
מפורש בלבד.

3

תרחיש הכשל
מנהל מנסה לחשוף זהות
מדווח, נחסם, והניסיון נרשם
בלוג.

זה חומר הגלם. היום מוסיפים את שכבת ההחלטות.

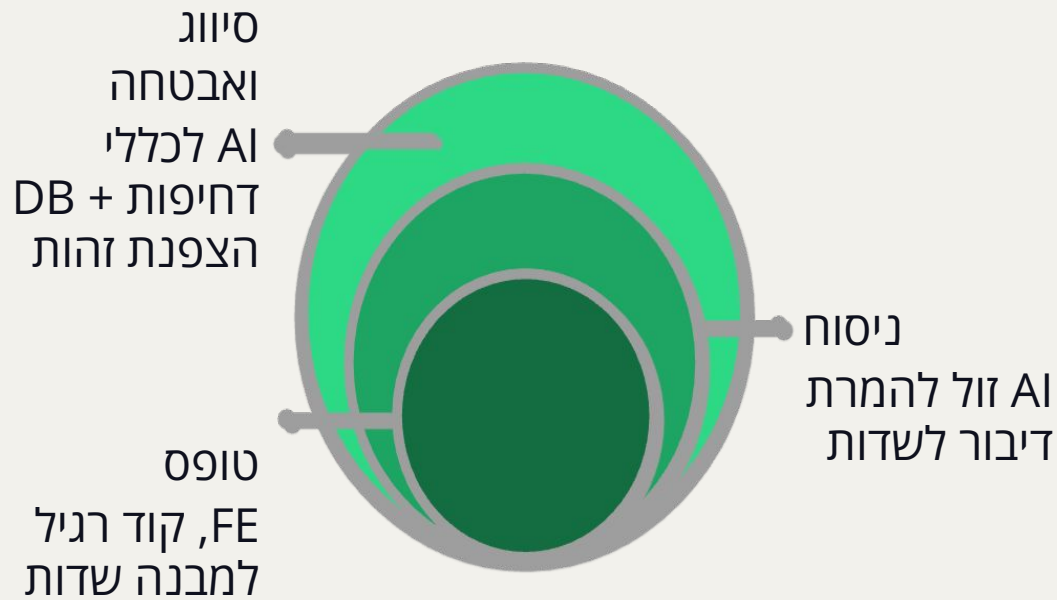


שכבת ההחלטות: חמש שאלות לכל דרישה

לפי סעיף 14 של ה-PRD, לכל דרישה עונים על:



דוגמה: דיווח תקלה מפורק להחלטות



ההחלטות שורה אחר שורה

טופס הדיווח: קוד רגיל, FE. אין סיבה לערב מודל.

ניסוח הדיווח מדיבור חופשי לשדות מובנים: AI, מודל זול מספיק. משימת ניסוח, לא הסקה.

סיווג דחיפות: AI עם כללי גבול בקוד רגיל. תקלה קריטית לעולם אינה תלויה רק במודל.

שמירת הדיווח והפרדת הזהות המוצפנת: קוד רגיל, DB, דרך Repository אחד.

כל שורה כזאת היא החלטת פיתוח מתועדת, לא תיאור תרחיש.

שלוש בדיקות לכל החלטה

בדיקת הצורך/ רצון
איזו דרישה מה-PRD היכולת משרתת? אם אין דרישה או סיפור
משתמש - לא נכנס למערכת.

בדיקת החלופה הפשוטה/ הזולה
קוד רגיל לפני AI, מודל זול לפני יקר, פתרון קיים לפני פיתוח.

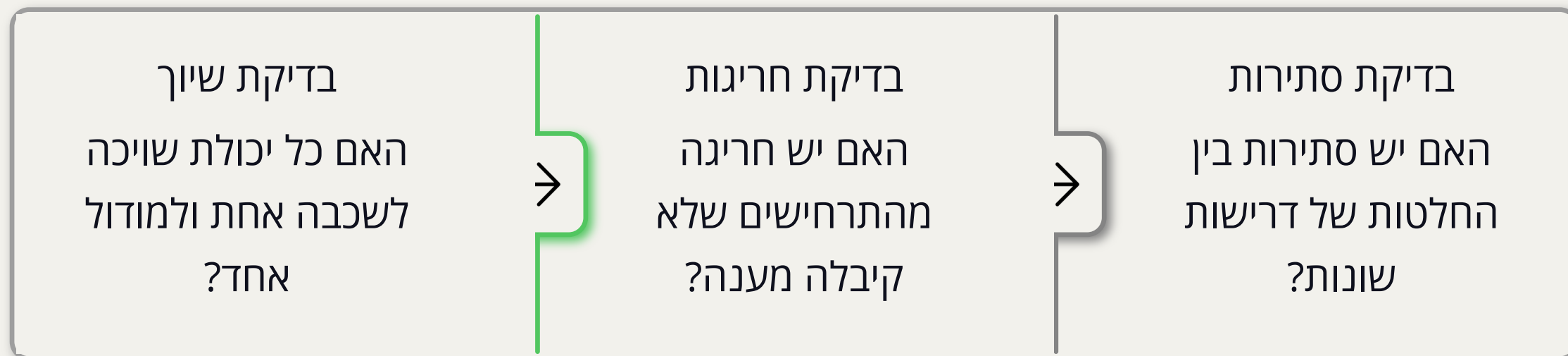
בדיקת העלות המתמשכת
כמה בחודש כולל תחזוקה, ומי הבעלים?

יכולת שנפלה בבדיקה מקבלת שורת דחייה מתועדת (ADR) עם
שלב היעד והנימוק. דחייה מתועדת היא תוצר לגיטימי כמו יישום.



לפני שממשיכים Self-Check

אחרי כתיבת החלטות הפיתוח לכל הדרישות, מריצים בדיקה עצמית:



מתקנים לפני שממשיכים לארכיטקטורה (סעיף 15).



חוט השני: מהתרחישים אל המערך

התרחישים, ובעיקר החריגים, הופכים למערך הזהב לבדיקות:
אשכולות נורמה, מקרי קצה וכשלים.

תרחישי קצה	תרחישי נורמה
מקרים גבוליים שנבדקים בקפידה	הזרימה הרגילה שעובדת כצפוי
תרחישי כשל	
מה קורה כשהמערכת נכשלת	

איכות מקרי השימוש היום קובעת את איכות הבדיקות מחר.

המנגנון:

כמו בכל נושא: יוצרים קודם את מסמך הפרויקט, אחר כך עדכון ב-PRD.

2	1
ריצת פרומפט עדכון מריצים את פרומפט העדכון בצ'ט "PRD", מסמך מתפתח".	שמירת המסמך מסמך מקרי השימוש נשמר במסמכי הפרויקט.
4	3
ריצת פרומפט הבדיקה מריצים את פרומפט הבדיקה: אין המצאות, אין סתירות.	מילוי סעיף 14 סעיף 14 מתמלא, ושורה נוספת ביומן השינויים.

נהלי עבודה

יכולת בלי נוהל היא בעייתית בחברות. הכלי מאפשר, הנוהל מקיים.

בעלות על קבצים

לכל קובץ בעלים אחד. שם קובץ הוא חוזה: תחילית קוד למכונה, מלל עברי חופשי לאדם.

האינדקס

המערכת מייצרת, האדם מפקח. שגיאה מתקנים במקור, לא באינדקס.

אמת אחת תמיד

הצעות מטופלות עם הגעתן, אישור גורר עדכון נגזרות מידי.

גרסאות קוד

Commit לכל שינוי לאחר ביצועו, אחד לנושא. מחיקה היא העברה לארכיון.



יצירה וניהול משתמשים

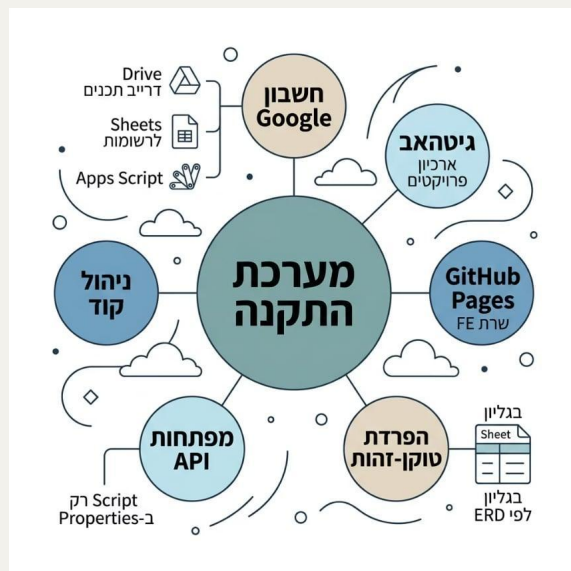
שלב 0 של תוכנית הבנייה, בידיים, על הפרויקט האמיתי:

עקרונות הבסיס

חשבון Google ייעודי לפרויקט: Drive לתוכן, Sheets לרשומות, Apps Script באותו חשבון.

מאגר GitHub ו-GitHub Pages: כלי אחד, שני תפקידים, ארכיון ושרת ה-FE.

הפרדת טוקן מזהות בגיליון, לפי ה-ERD: עקרון האנונימיות נשמר כבר ברמת המשתמשים וההרשאות.



כלל הסודות: מפתחות API שייכים תמיד לשכבת המחליט, ב-Script Properties, לעולם לא בדפדפן.

עכשיו תורכם

תוצר השיעור

מסמך מקרי שימוש
והחלטות פיתוח לשמונה
הדרישות, בנוי כשכבה
מעל התרחישים הקיימים.

תרגול

השלמת ההחלטות
לדרישות שלא הוספקו
בכיתה, ריצת עדכון בצ'ט
ה-PRD, וריצת פרומפט
הבדיקה.

בהמשך

מהחלטות הפיתוח אל
הנתונים והמסכים,
סעיפים 15 ו-16, פירוק
ארכיטקטוני מלא.

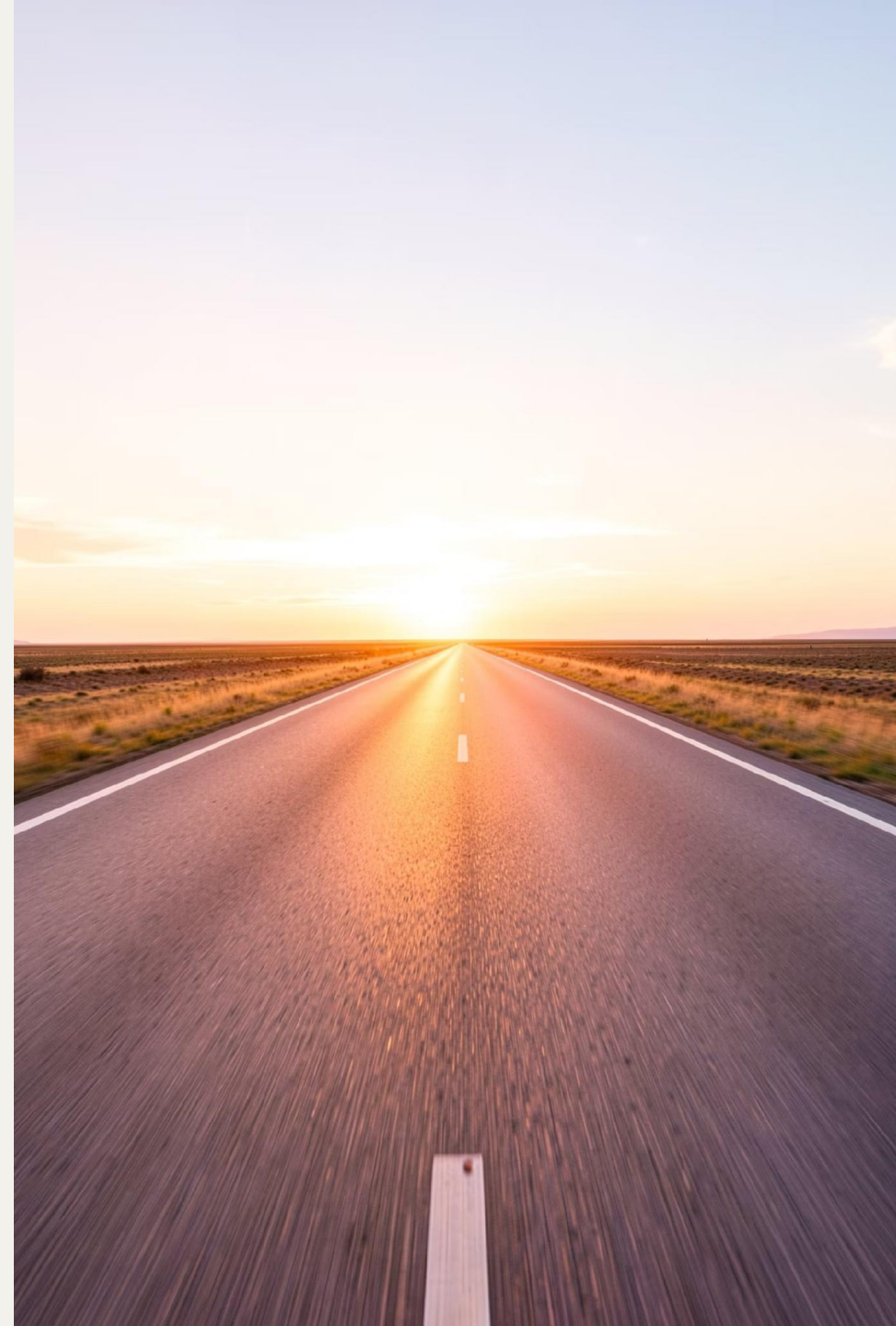
השורה התחתונה

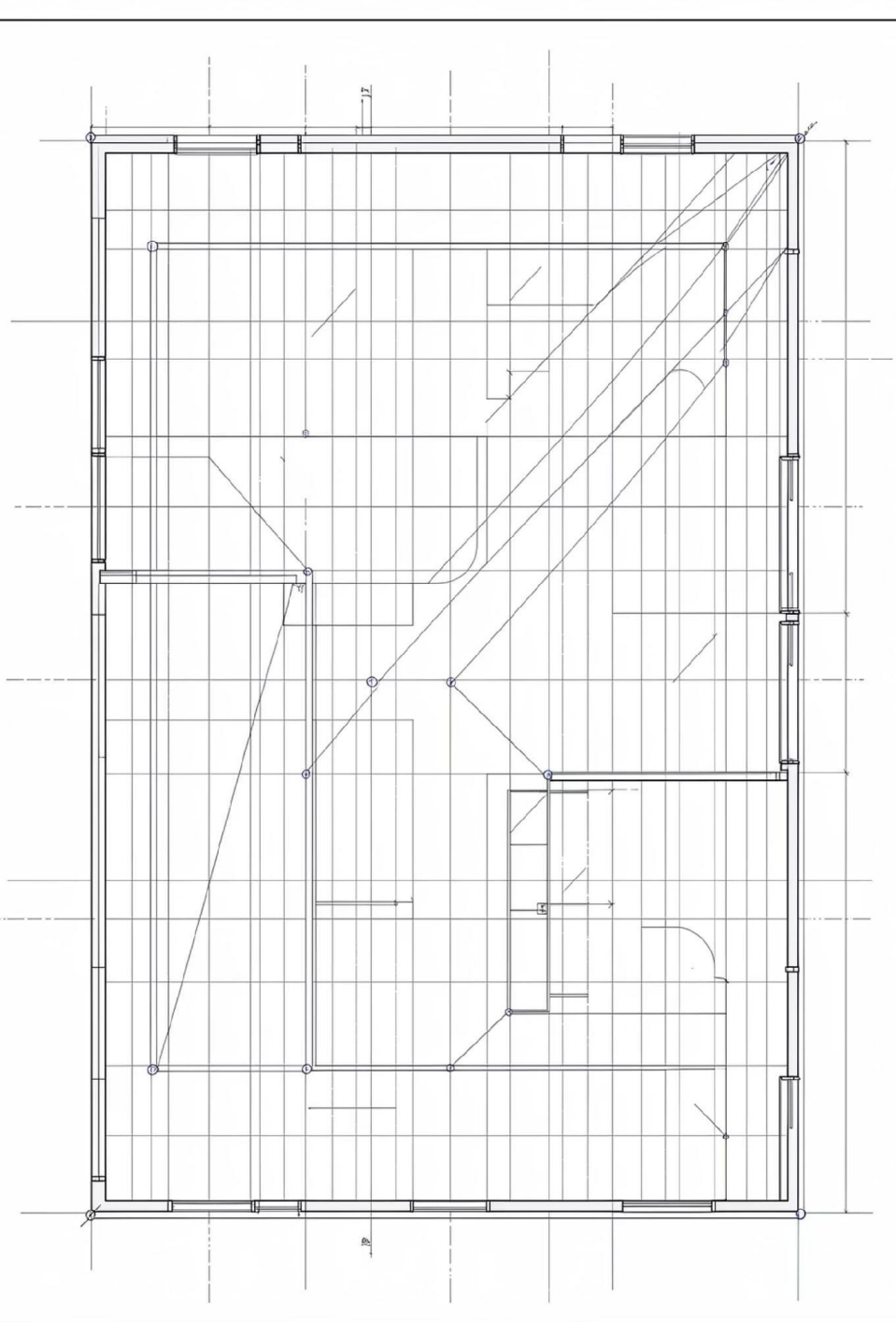
כל פונקציונליות
פיצול לשכבות, הגדרת
המודולים להם נותנים
שירות, בדיקה אחת
לפחות.

התרחישים נכתבו
ההחלטות נכתנו.

הדרך קדימה
המסמך נשמר, ה-PRD
מתעדכן, והדרך
לארכיטקטורה פתוחה.

סדר העדיפויות
קוד רגיל לפני AI, מודל
זול לפני יקר, ודחייה
מתועדת שווה יישום.





שיעור 12, חלק ב: תוכנית הבנייה איפה הכול יושב

מאב טיפוס לאפליקציה אמיתית
קורס מיישם AI, האקריו

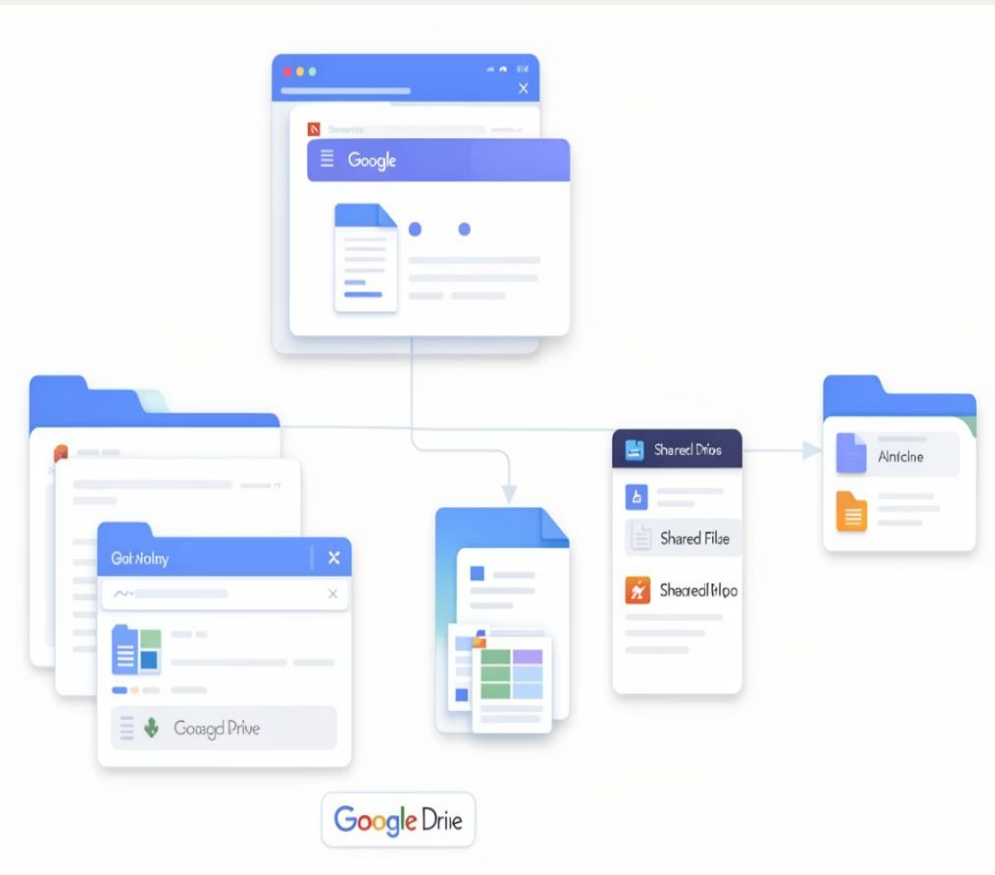
נקודת המוצא

יש לנו אב טיפוס HTML עובד, ישנם תכנים. עכשיו צריך להגדיר ארכיטקטורה:



שאלה 1: איפה שומרים?

חלוקה לפי סוג התוכן, בחשבון Google.



רשומות 📊

אינדקס והתקדמות:
בגיליון Sheets ייעודי

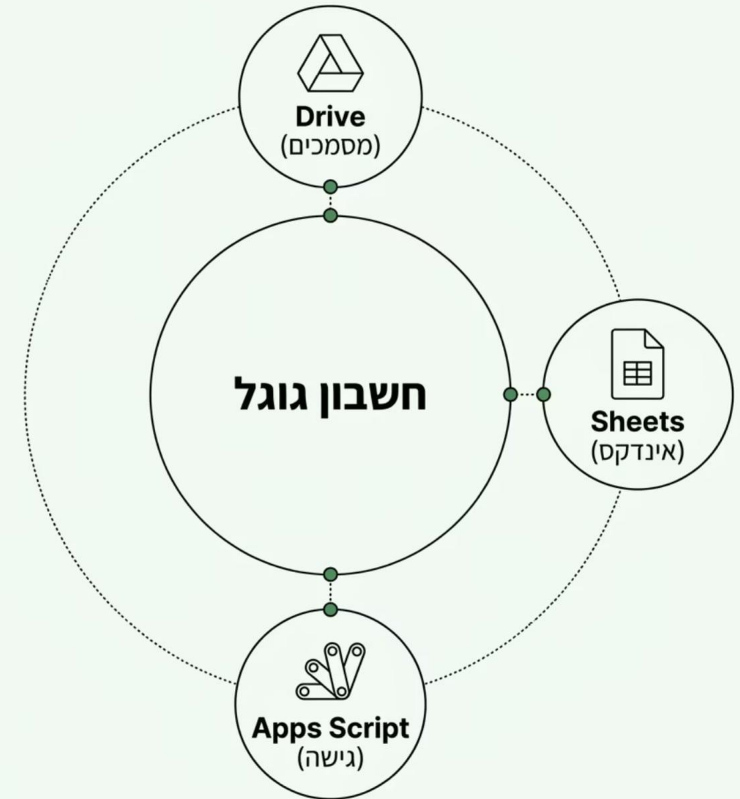
מסמכים 📄

תמלול ותקציר,
בתיקייה ב Drive
במבנה קבוע, תיקייה
לכל נושא.

שאלה 2: מה זה גיליון האינדקס?

גיליון שמכיל שורה לכל פריט, עם מזהי
קבצים קבועים.
חיפוש מידע - האינדקס יודע באיזה קובץ
לחפש.

הקסם: הכול בחשבון Google.
הגישה מ-Apps Script מובנית, בלי
מפתחות ובלי הרשאות.



שאלה 3: מה יש במערכת?

מה יש ✓

מה לא שומרים ✗

אינדקס

קוד הלוגיקה

קוד המסכים

קבצים כבדים

מפתח Gemini
BE-ב

נתוני משתמש

טקסטים
מעובדים

הקלטות

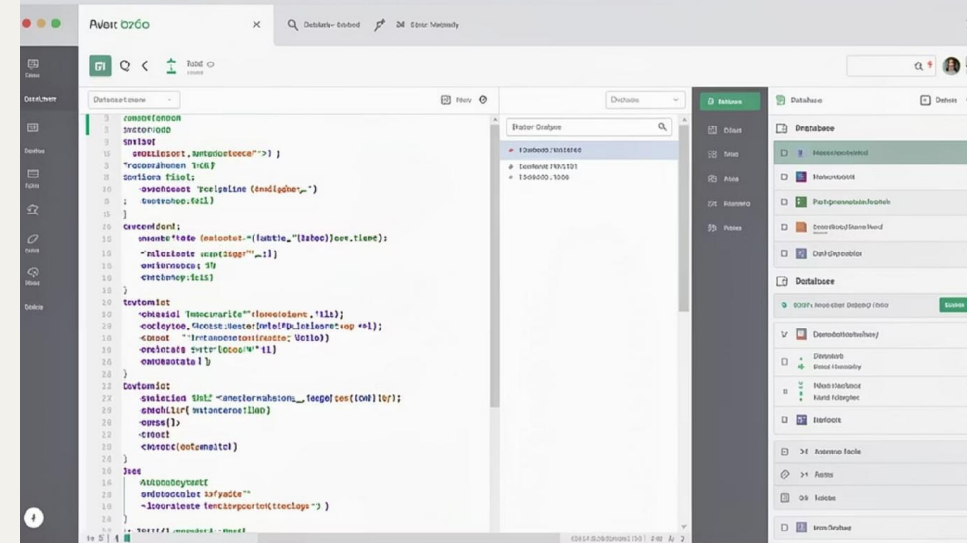
מקורות מידע גולמיים

המערכת היא קוד, אינדקס וטקסט. כל השאר נשאר בחוץ.

שאלה 4: איפה נבנה את המערכת?

נבנה את המערכת כולה ב Apps Script:
HTML, קוד, גיליון (DB)

ניהול הקוד מתבצע לרוב ב GitHub. אותו נלמד
בשיעור Claude Code, ככלי ניהול הגרסאות
והפיקוח על הסוכן.





שאלה 5 - איפה ה-BE?

שתי אפשרויות לשכבת ה"מחליט":

Google Apps Script
גישה מובנית ל-Drive ול-Sheets,
מפתח בכספת, אפס התקנות.

Serverless בענן
מקצועי וקרוב לעולם העבודה,
דורש הגדרת הרשאות.

ההכרעה לשלב הזה

שדרוג עתידי

Apps Script

כל הנכסים חיים בחשבון Google אחד, ו-Apps Script הוא הדרך היחידה שבה כולם נגישים בלי תשתית הרשאות.

01

פונקציה

כותבים את הלוגיקה
ב-Apps Script.

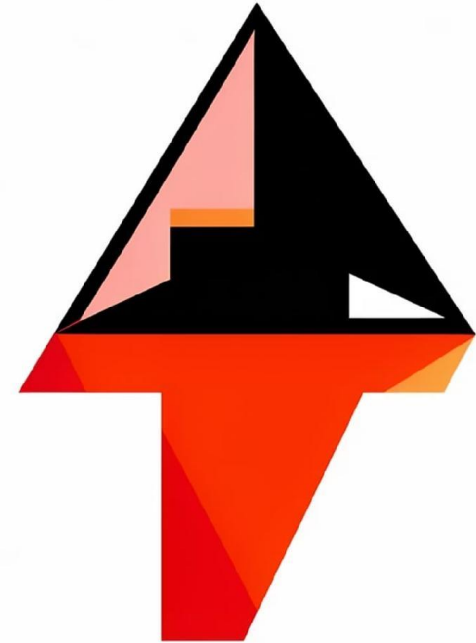
03

BE

מערכת עובדת ללא תשתית נוספת.

02

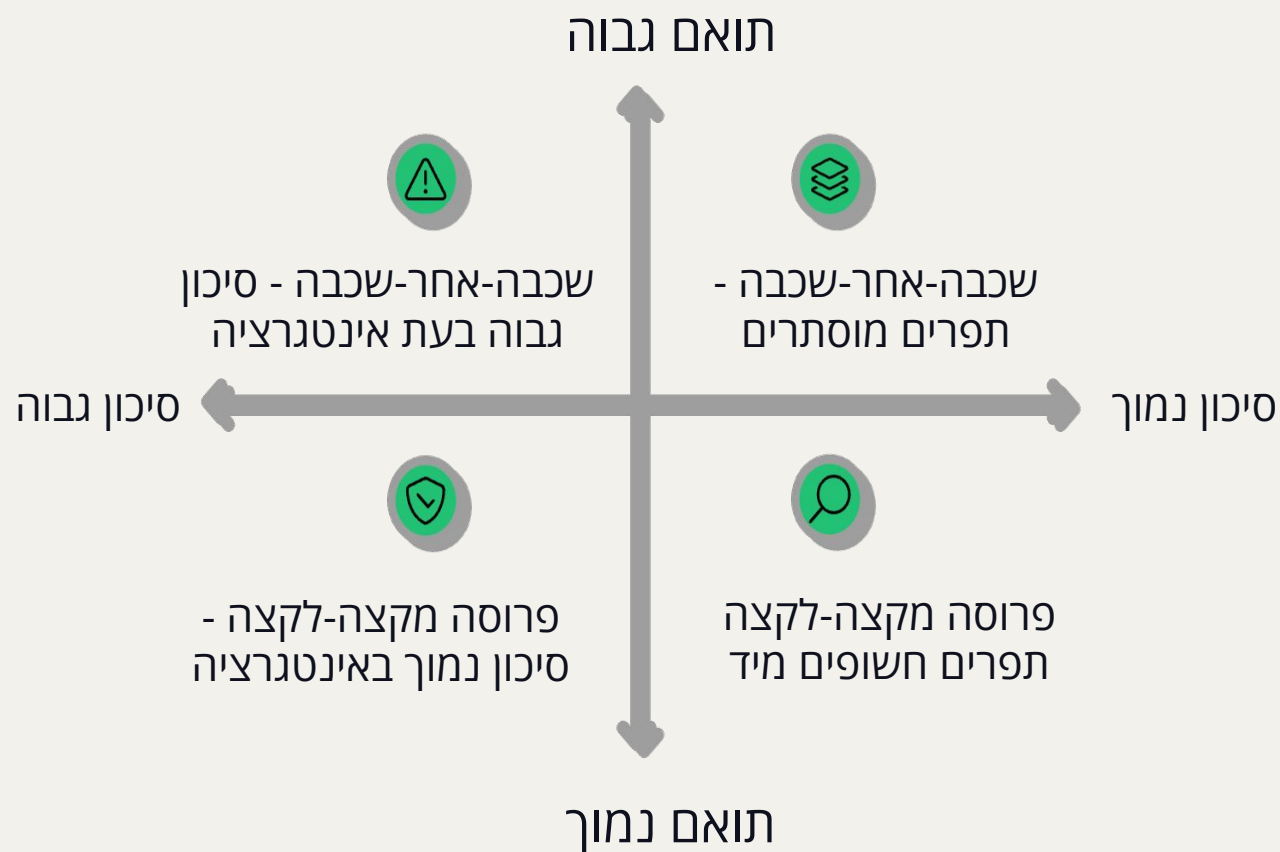
פרסום כ-Web App
חושפים את הפונקציה
כנקודת קצה.



עיקרון הבנייה: שכבה אחר שכבה או פרוסה מקצה לקצה?

למה לא לבנות כל שכבה בנפרד?

הסיכון נמצא בתפרים שבין השכבות:



- הרשאות
- פורמט JSON
- פריסה
- מפתחות

פרוסה חוצה את כל השכבות - מטפלת בתפרים, בכל תרחיש



מה זו פרוסה?

תרחיש אחד, מהמסך ועד לבסיס הנתונים, דרך כל השכבות.

כמו פרוסת עוגה: חיתוך שטועמים בו את כל השכבות בביס אחד.

הפרוסה שלנו

יצירת דיווח - מהמסך ועד הגיליון.

כשהיא עובדת

שלד מהלך (Walking Skeleton) - מערכת מינימלית שחיה.

פרוסות לעומת מונולית

פרוסה היא יכולת דקה בהיקף שלה, לא במבנה. בתוכה יש ארכיטקטורה מלאה:



שמנתב doPost

העובר של

האורקסטרטור - מנתב

בקשות לפונקציות

הנכונות.



פונקציית Gemini
נפרדת

שירות ה-AI - מבודד

ומוחלף בקלות.



פונקציית כתיבה נפרדת

ה-Repository - כותב

לגיליון, ניתן להחלפה.

מבחן ההחלפה: בהמשך נחליף את הגיליון במסד נתונים - כמה פונקציות משתנות? אחת = מודולרי.

שלב 0: תשתית

ניצור פרויקט Apps Script חדש
בחשבון ה Google שלנו

תוך דקות בנינו סביבת עבודה מלאה:
עורך, אירוח, גרסאות, כספת.



שלב 1: ה-DB

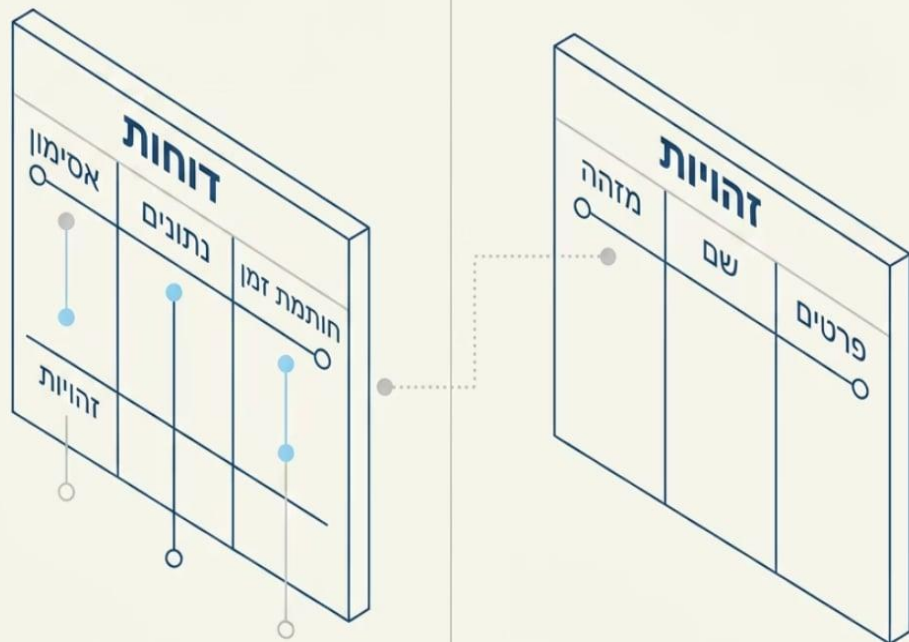
נתחבר לגיליון עם שתי לשוניות לפי ה-ERD
ניצור אותו עם CLAUDE

reports

עם טוקן מדווח, לא
שם - הפרדת הזהויות
מהשורה הראשונה.

identities

בנפרד - הפרדה
נקייה בין זהות
לדיווח.



שלב 2: ה-BE

פרויקט Apps Script עם שלושה רכיבים:

doPost
מקבל דיווח וכותב לגיליון.

קריאה ל- KEY
מהשרת - לא מהדפדפן.

מפתח נשמר ב-Script Properties
מאובטח בצד השרת.

ה-KEY הסודי עוזב את הדפדפן ולעולם אינו מופיע שם.





שלב 3: החיבור

וה-FE מוגש ישירות מ-Apps Script (HtmlService):

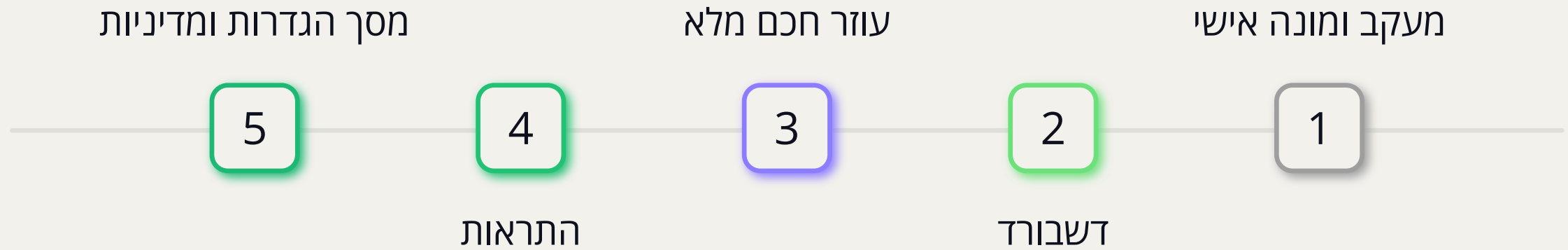
אפס כלים חיצוניים

מסך ומוח יחד

אותה כתובת

שלב 4: הרחבה מודול אחר מודול

כל מודול ייבנה באותו דפוס, על שלד שכבר עובד:



לכל צעד Commit

רשת ביטחון לכל שינוי.



גישה דרך ה-Repository בלבד
הפרדה נקייה בין שכבות.



למה גוגל?

אין כלי הכי טוב, יש כלי שמתאים למשימה.

המשימה שלנו הגדירה את הבחירה:



חיכוך מינימלי
הגדרות מינימליות, גישה
מיידית.



חינמי לגמרי
אפס עלות תשתית לאורך
כל הקורס.



שקוף ללומד
ניתן לראות ולהבין כל שכבה.

סביבת גוגל מתאימה לתנאים, ולכן נבחרה.

מה עוד קיים ונלמד בהמשך?

כלי No-Code 
מסחריים

פתרונות ויזואליים ללא
קוד.

פלטפורמות 
מקצועיות

- Supabase, Vercel
לפרויקטים בסקייל.

סוכני קוד 

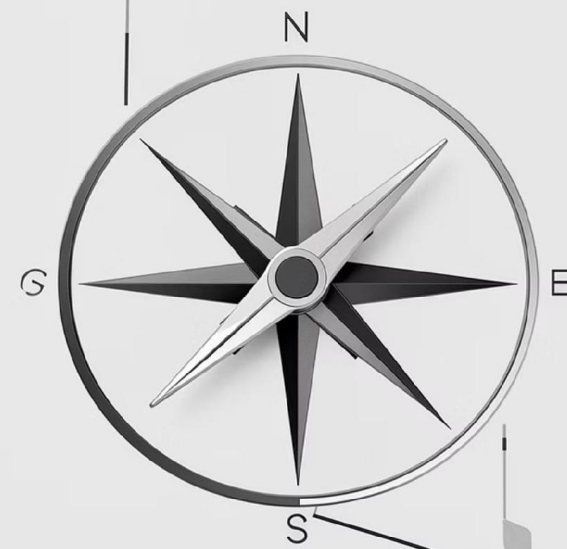
Claude Code, Codex
- כותבים ומריצים קוד
בעצמם.

המתודולוגיה שלמדנו היא בדיוק מה שצריך כדי להנחות אותם נכון.

המסר: הכלי מתחלף, החשיבה נשארת

מי שמבין את
החלוקה יבנה נכון
בכל סביבה.

מי שלא, ישלם 75
דולר בכל סביבה.



סיכום: ארבע אבני דרך



החלפת האחסון דרך
ה-Repository
מעבר למסד אמיתי.



מודולים מצטרפים
כל מודול באותו דפוס תמיד
עובד.



שלד
הפרוסה מקצה לקצה, הסוד
מאחור.



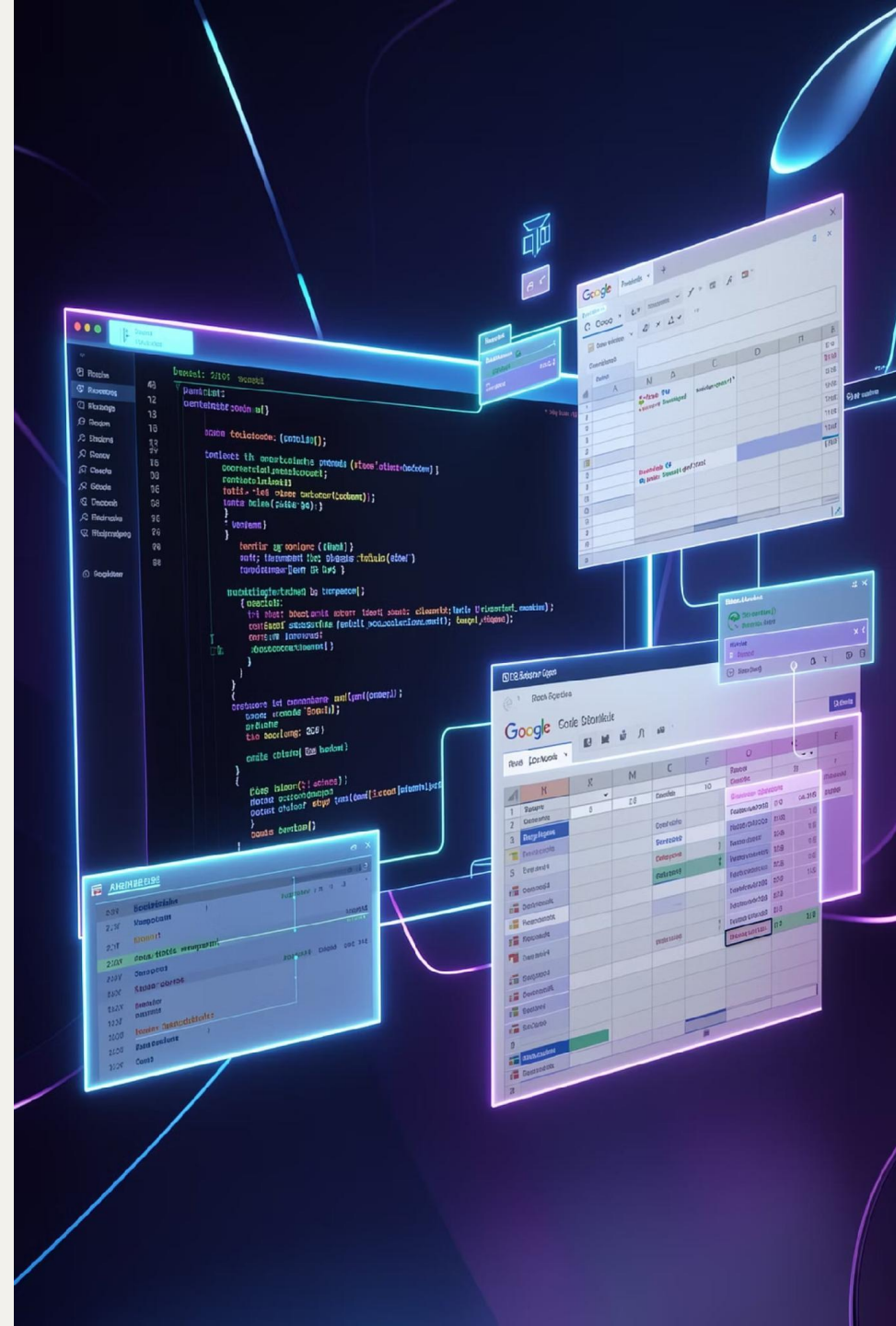
אבטחה וייצור
הכנה לסביבת Production אמיתית.

שיעור 12, חלק ג: Apps Script

מהצ'יט למערכת חיה

האקריז

קורס מיישם AI





מה זה Apps Script בשפה שלנו

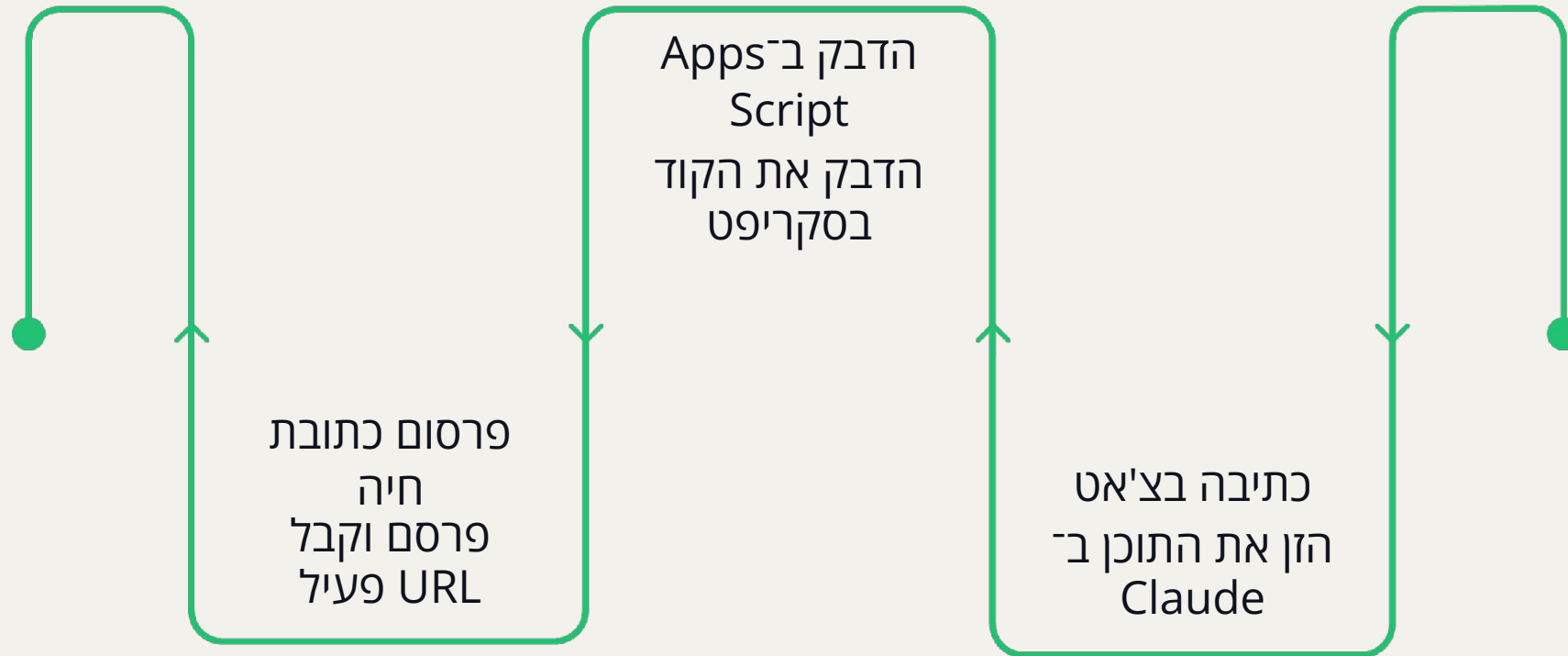
כל שלוש השכבות בכלי אחד, בחשבון גוגל, בדפדפן, בחינם.

קובץ Code.gs
המחליט - הלוגיקה שמאחורי
הקלעים

קובץ HTML
המציג - מה שהמשתמש
רואה בדפדפן

גיליון SHEETS
הזוכר - שמירת הנתונים

המסלול: מהצ'ט לכתובת חיה



שני כלים, אפס טרמינל. האדם הוא הידיים, הצ'ט הוא המוח, והתלמיד מבין כל צעד כי עשה אותו בעצמו.

צעד 1: יוצרים פרויקט חדש

2

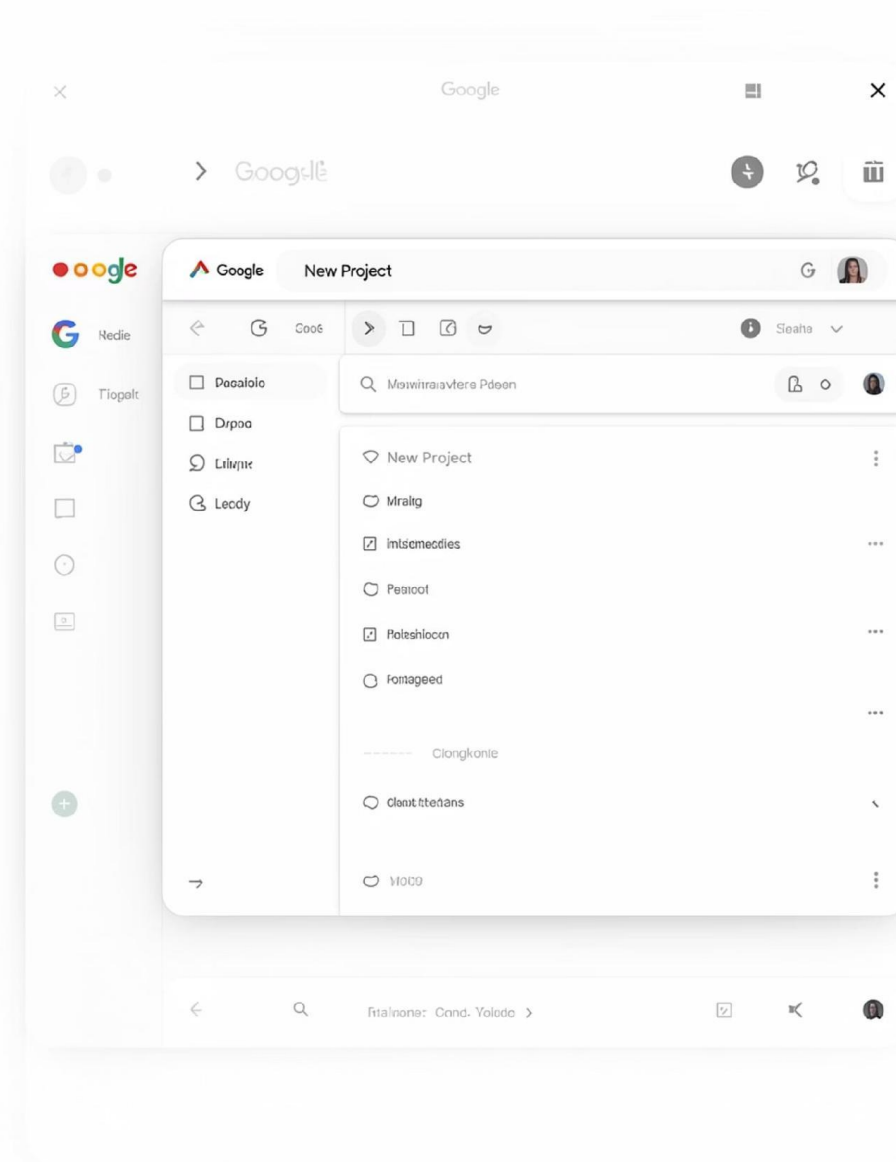
פרויקט חדש
לוחצים על "פרויקט
חדש" בממשק

1

פותחים
`script.google.com`
בחשבון הייעודי של
המערכת

3

שם לפי כללי השמות
מתן שם ברור ועקבי לפרויקט



צעד 2: מבקשים מקלוד בצ'ט

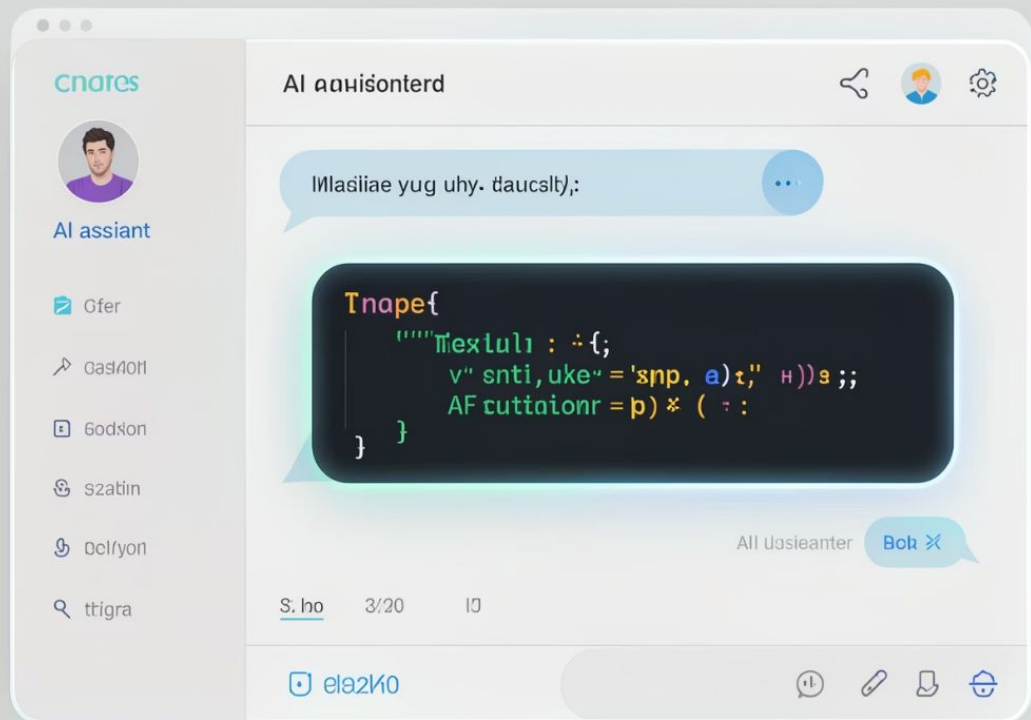
משפט ההקשר הקבוע

"הקוד ירוץ ב-

Google Apps Script ויוגש כ-Web App.
הפרד קובץ HTML (תצוגה) מקובץ Code.gs
(לוגיקה)"

צור הפרדה בין השכבות, המודולים והפונקציות

ההפרדה מוגדרת מראש בפרומפט - כך קלוד
יודע בדיוק איך לבנות את הקוד, ומקבלים פלט
מסודר ומוכן להדבקה.



צעד 3: מדביקים

Code.gs

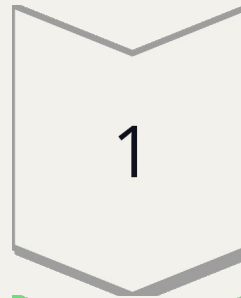
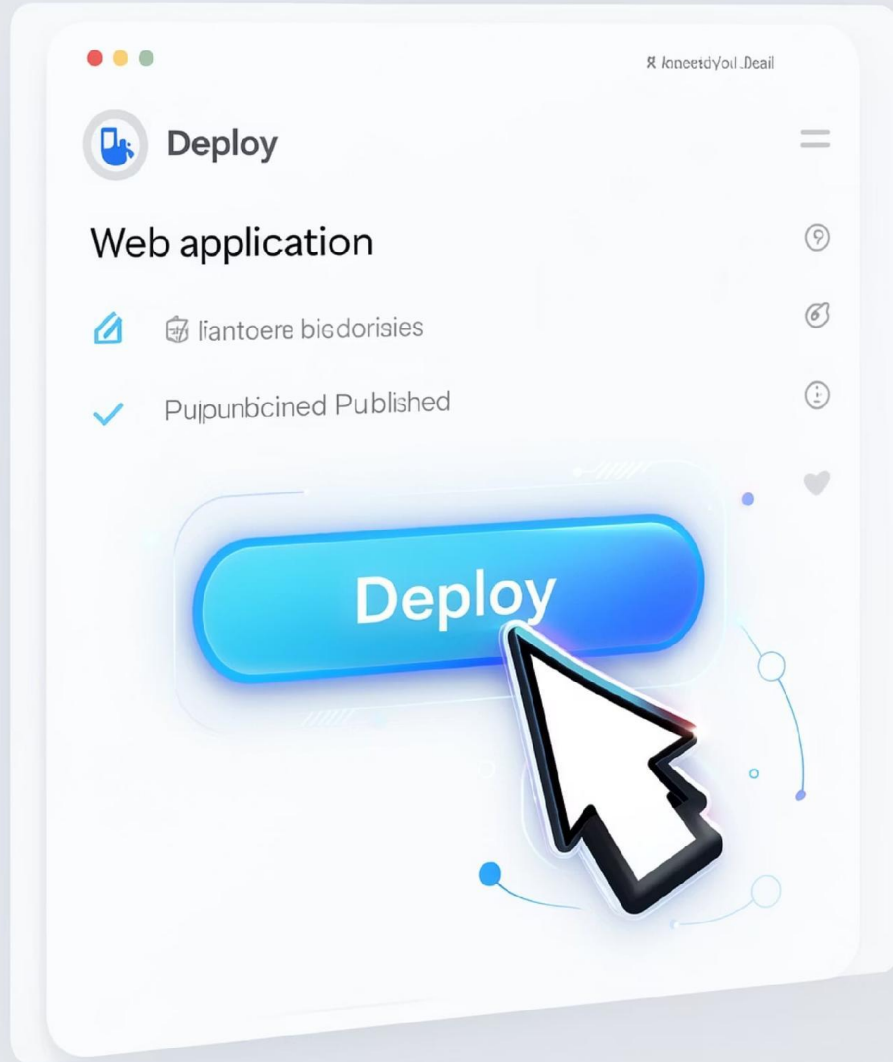
קודי הלוגיקה. הלב הפועם של המערכת. כאן נמצאות כל ההחלטות, הפונקציות, והגישה לגיליונות (בסיס נתונים).

קבצי HTML

ממשק המשתמש - מה שהמשתמש רואה ומקיים איתו אינטראקציה בדפדפן.

שלוש שכבות, קבצים, ההפרדה הופכת את הקוד לקריא ולניתן לתחזוקה.

צעד 4: מפרסמים



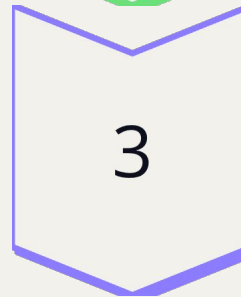
Deploy כ-Web App

בוחרים "פרסם כ-Web App" מתפריט הפרסום



בוחרים רמת גישה

מי יכול להשתמש במערכת - ארגון, כולם, או רק אתם



מקבלים כתובת

כתובת אינטרנט חיה ופעילה מיידית

שלושה הרגלים

1

מפתח רק בכספת
אף פעם לא בקוד.
יש שתי כספות: מפתח של
המערכת ב-Script Properties
(אחד, של בעל המערכת),
ומפתח אישי של משתמש
ב-UserProperties (כספת
נפרדת לכל מחובר).

2

לפני שינוי גדול
שמירת גרסה בשם.

3

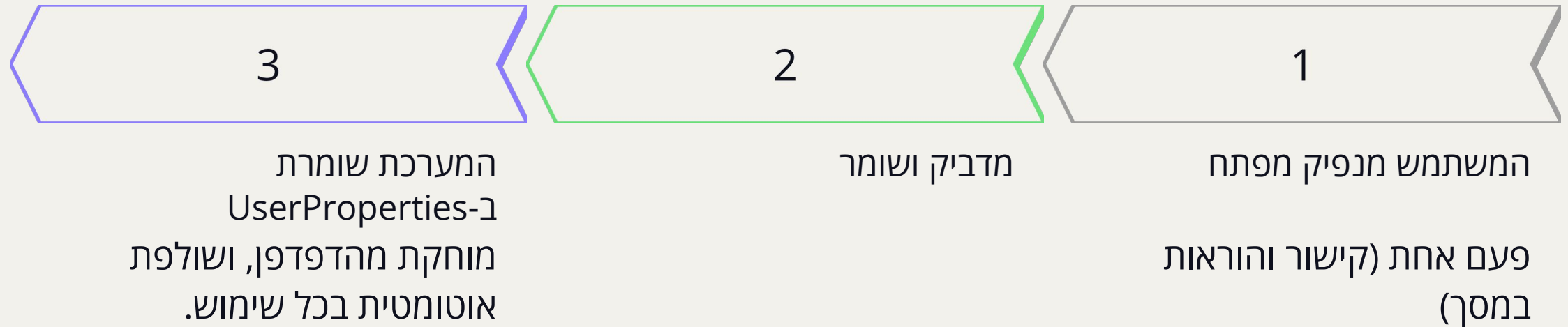
נתונים דרך פונקציה אחת בלבד

כל משתמש והמפתח שלו (BYOK)

כשכל משתמש מביא מפתח Gemini משלו, כל אחד רץ על המכסה החינמית שלו.

מאה משתמשים = מאה מכסות, והמכפלה עובדת לטובתנו.

הזרימה:



הכלל

המפתח עובר בדפדפן פעם אחת בחיים, בדרך לכספת.

לא מוצג

לא מוחזר

לא נשמר ב-HTML

אחרי השמירה המסך מציג רק:

✓ מפתח שמור

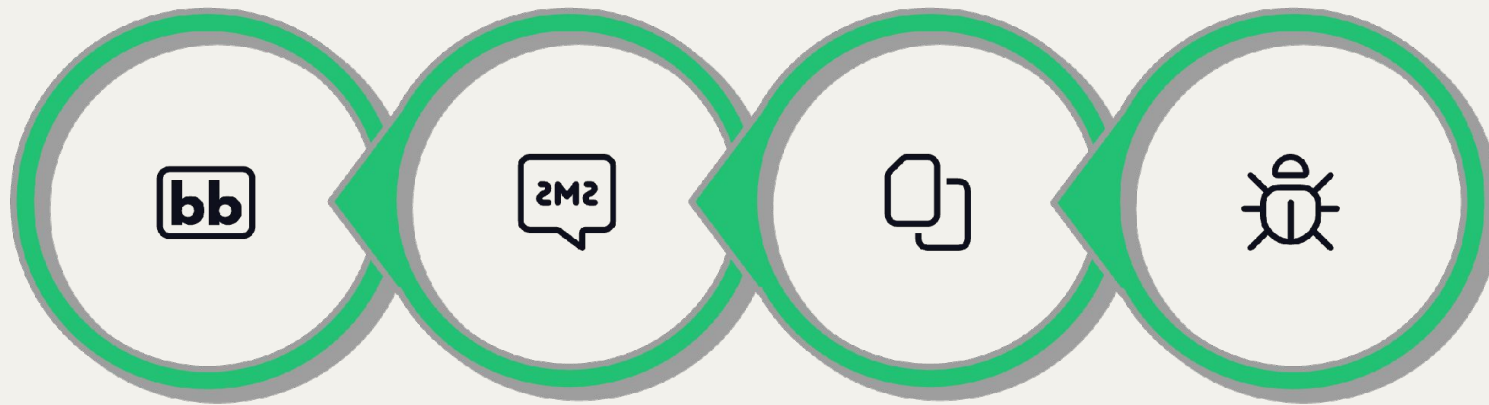


צעד 5: יש מערכת.

המערכת חיה

שולחים את הקישור, וכל אחד משתמש בדפדפן. אין צורך בהתקנות, אין צורך בהגדרות מיוחדות - רק קישור ודפדפן.

וכשמהו לא עובד



הדבק ופרוס
מחדש

קבל תיקון

העתק הודעת
שגיאה

שגיאה מופיעה

הלולאה: בקשה ← הדבקה ← שגיאה ← תיקון ← הדבקה
אתם מנהלים את הסוכן, לא מקלידים או יוצרים קוד.

שלושה הרגלים

1	כלל 2: Script Properties מפתח רק ב-Script Properties, אף פעם לא בקוד. מפתחות ופרטים רגישים לא נכנסים לקוד לעולם.
2	רשת הביטחון לפני שינוי גדול: שמירת גרסה בשם. תמיד אפשר לחזור אחורה.
3	כלל 3: Repository ראשון נתונים רק דרך פונקציה אחת שכותבת לגיליון.

B Kogeni Habit's

- Hsrrmen trotaiong be Rudiits

- Connpicess No Rubies
- Goaldienab Buiets

רטרוספקטיבה

שלוש שאלות לסיום

1

מה למדתי היום שלא ידעתי?

תובנה חדשה שמשנה את
האופן שבו אני חושב על ניהול
מוצר.

2

מה הפתיע אותי?

משהו שלא ציפיתי לו — תפיסה
שהתהפכה, הנחה שנשברה.

3

מה אקח איתי כבר מחר?

פעולה קונקרטית אחת שאוכל
ליישם מחר בבוקר בעבודה.

תודה רבה

רילי לבבי

053-4365200

levavir@gmail.com

